

Unit 1: Overview of Web Technology (4 hrs.)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Contents

- 1.1 Introduction to WWW, URL, HTTP, HTTPS
- 1.2 The Evolution of Web Technologies
- 1.3 Website and Web Application
- 1.4 Web Client and Web Server
- 1.5 HTTP Request and Response
- 1.6 Client side and server-side scripting language
- 1.7 DNS and its hierarchy
- 1.8 Web Hosting

Introduction to WWW

- The World Wide Web (WWW) is a vast information system that is built on hypertext documents that are linked together. It is accessed over the Internet using web browsers (such as Chrome, Firefox, Safari).
- The WWW allows users to access text, images, videos, and other resources via hyperlinks.
- **Example:** When you open a browser and enter a website address like <https://www.nast.edu.np>, you are using the WWW to access information stored on a server.

Introduction to WWW

World Wide Web (WWW) is:

- A system of interconnected web pages
- Uses the Internet to access resources
- Developed by Tim Berners-Lee in 1989
- Web pages are connected using hyperlinks

Components of WWW

Main Components

- Web Browser
- Web Server
- Web Pages
- Hyperlinks
- Internet Protocols

URL (Uniform Resource Locator)

URL

- A URL is the address of a resource on the web.

Structure of URL

<https://www.nepalstock.com.np/company/detail/131>

<https://nast.edu.np/>

Components

- Protocol → HTTPS
- Domain Name → nepalstock.com.np
- Path → /company/detail/131

HTTP (HyperText Transfer Protocol)

HTTP

- Protocol used for communication between browser and server
- Transfers web pages and web resources
- Stateless protocol

Features

- Request-response model
- Uses port 80
- Fast communication

Example

- Opening: <http://example.com>

HTTPS (HyperText Transfer Protocol Secure)

HTTPS

- Secure version of HTTP
- Encrypts communication using SSL/TLS
- Protects:
 - Passwords
 - Banking information
 - Personal data
- **Uses Port 443**

Example

- [eSewa](#)
- [Khalti](#)
- [NIC Asia Bank](#)

HTTP vs HTTPS

Feature	HTTP	HTTPS
Security	No encryption	Encrypted
Port	80	443
SSL Certificate	Not required	Required
Data Safety	Low	High
Use Case	Informational websites	Banking, e-commerce

Evolution of Web Technologies

Web 1.0 (Static Web)

- Read-only websites
- Static HTML pages
- In the early days of the internet, websites were made up of simple HTML pages. These pages were static, meaning the content did not change unless manually updated by the website owner.
- **Example:** A personal blog from the 1990s or early 2000s would often consist of static HTML pages, where text and images were displayed without interaction.

Evolution of Web Technologies

Web 2.0 (Interactive Web)

- User interaction
- Social media
- Dynamic content
- Web 2.0 introduced dynamic content and interactivity. The use of JavaScript, CSS for styling, and AJAX (Asynchronous JavaScript and XML) allowed websites to be more interactive and real-time. Websites could now update content without requiring a full page reload.
- **Example:** Social media platforms like Facebook, Twitter, or Gmail, which dynamically update content (such as posts, messages, etc.) without requiring users to refresh the page.

Evolution of Web Technologies

Web 3.0 (Semantic & Intelligent Web)

- AI integration
- Personalized services
- Smart systems
- Web 3.0 focuses on decentralized applications, often using blockchain technology. Web apps are more intelligent, making use of artificial intelligence (AI) and machine learning.
- **Progressive Web Apps (PWA)** provide native-like mobile experiences, allowing users to install websites on their devices and use them offline.
- **Examples:** AI chatbots, Recommendation systems, Decentralized finance (DeFi) applications using blockchain technology, and PWAs like Instagram, which can function both as a web app and a mobile app.

Website vs Web Application

Website	Web Application
Mainly informational	Interactive
Static content	Dynamic content
Limited user interaction	High user interaction
Easier to develop	More complex

Examples

Website

- [Visit Nepal](#)

Web Application

- [Citizen Portal Nepal](#)
- PU Exam Form Portal

Web Client

Web Client

- A client requests services from the server.

Examples

- Web browsers
- Mobile apps

Functions

- Sends requests
- Receives responses
- Displays content
- Student accessing PU result through mobile browser

Web Server

Web Server

- A server stores and delivers web resources.

Functions

- Processes requests
- Sends responses
- Hosts websites

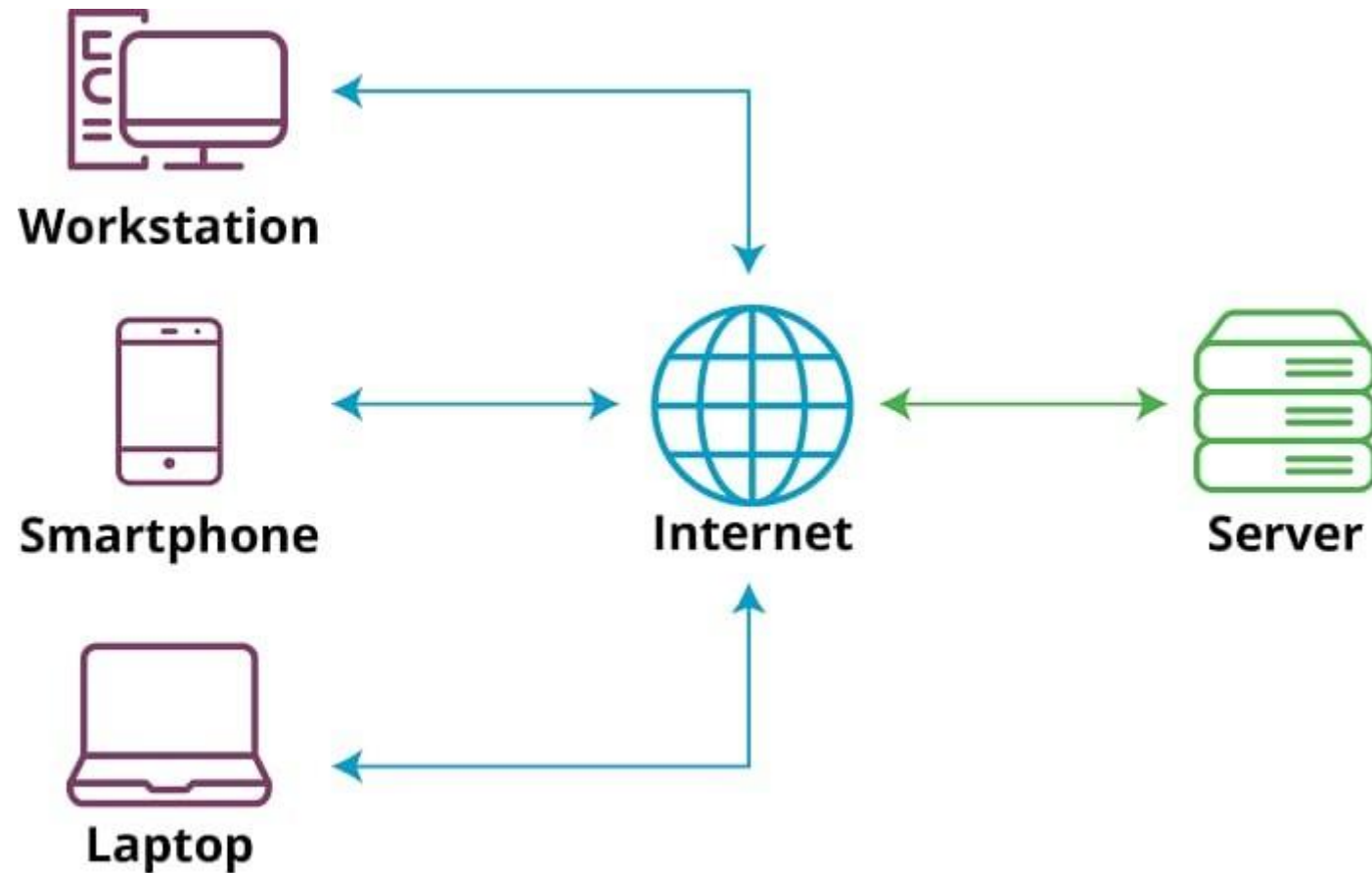
Popular Web Servers

- Apache
- Nginx
- Microsoft IIS

Example

- Servers hosting:
- eSewa
- Khalti
- Government portals

Client-Server Architecture



Client-Server Architecture Workflow

Example

- Student opens: <https://exam.pu.edu.np:9094/>

Process

- Browser sends request
- Server processes request
- Server sends webpage
- Browser displays result

Components of HTTP Request and Response

HTTP Request

- **HTTP Method:** This tells the server what action to perform (e.g., GET for retrieving data, POST for submitting data).
- **URL/URI:** The Uniform Resource Identifier, which specifies the location or resource you want to interact with.
- **Headers:** These provide additional information about the request, such as the browser type, accepted content types, and more.
- **Body:** In some requests, like POST or PUT, you can include data (e.g., form data or JSON) that the server should process.

Components of HTTP Request and Response

HTTP Response

- **Status Code:** A three-digit number indicating the result of the request (e.g., 200 for success, 404 for not found, 500 for server error).
- **Headers:** Similar to request headers, these provide additional information about the response, like content type, server information, and caching directives.
- **Body:** The actual content of the response, which can be HTML for web pages, JSON for data, or other types of data based on the request and server's purpose.
- The response includes a content-type header specifying that the content is HTML, and the body contains an HTML web page.

HTTP Method Types

HTTP methods help your #computer and the internet work together by telling servers what you want to do with the information or resources they have.

- **GET:** Like asking for information. When you type a web address in your browser and hit Enter, you're making a GET request to see a webpage.
- **POST:** Like submitting a form. When you fill out an online form and click "Submit," you're sending a POST request with your data.
- **PUT:** Like updating information. If you want to change your username on a website, you might use PUT to tell the server your new username.

HTTP Method Types

- **DELETE:** Like removing something. If you want to delete a post or a file, you'd use DELETE to ask the server to get rid of it.
- **HEAD:** Like checking if something exists. You can use HEAD to ask the server if a web page or file is there, but you don't need the actual content.
- **OPTIONS:** Like asking what you can do. It's like checking with the server to see what methods (GET, POST, etc.) are allowed for a resource.

Example of a HTTP Request and Response

HTTP Request

- **Method:** GET
- **URL/URI:** <https://www.nast.edu.np/index.html>
- **Headers:** User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.0.0 Safari/537.36
- **Accept-Language:** en-US,en;q=0.9
- **Body:** (There's usually no body in a GET request)

Example of a HTTP Request and Response

HTTP Response

- **Status Code:** 200 OK
- **Headers:** Content-Type: text/html
- **Server:** Apache/2.4.41 (Unix)
- **Body:**

Example of a HTTP Request and Response

```
<!DOCTYPE html>
<html>
<head>
  <title>Welcome to NAST</title>
</head>
<body>
  <h1>Hello, Students!</h1>
  <p>This is a simple Welcome page.</p>
</body>
</html>
```

Example of a HTTP Request and Response

In this example:

- The client (browser) sends a GET request to <https://www.nast.edu.np/index.html>
- The server responds with a 200 OK status code, indicating success.
- The response includes a content-type header specifying that the content is HTML, and the body contains an HTML web page.

HTTP Status Codes

- HTTP status codes are three-digit integers returned by a web server to indicate the outcome of a client's request.
- They are standardized by the IANA Status Code Registry and help developers and browsers understand if a request succeeded, failed, or requires further action

HTTP Status Codes

- Status Code Categories
- Codes are grouped into five classes based on their first digit

Range	Category	Description
1xx	Informational	Request received; the process is continuing.
2xx	Success	The action was successfully received, understood, and accepted.
3xx	Redirection	Further action must be taken to complete the request (e.g., a URL has moved).
4xx	Client Error	The request contains bad syntax or cannot be fulfilled (the client's fault).
5xx	Server Error	The server failed to fulfill an apparently valid request (the server's fault).

Commonly Encountered Codes

1xx - Informational

- 100 Continue: The server has received the request headers and the client should proceed to send the body.
- 101 Switching Protocols: The requester has asked the server to switch protocols.

2xx: Success

- 200 OK: The standard response for successful HTTP requests.
- 201 Created: The request was fulfilled, resulting in a new resource.
- 204 No Content: Request succeeded, but there is no content to send back.

3xx: Redirection

- 301 Moved Permanently: This and all future requests should be directed to the given URI.
- 302 Found: The resource is temporarily located at a different URI.
- 304 Not Modified: Tells the client to use the cached version of the resource.

Commonly Encountered Codes

4xx: Client Errors

- 400 Bad Request: The server cannot process the request due to a client error (e.g., malformed syntax).
- 401 Unauthorized: Authentication is required and has failed or not yet been provided.
- 403 Forbidden: The server understands the request but refuses to authorize it.
- 404 Not Found: The requested resource could not be found.
- 429 Too Many Requests: The user has sent too many requests in a given amount of time (rate limiting).

5xx: Server Errors

- 500 Internal Server Error: A generic error message when an unexpected condition was encountered.
- 502 Bad Gateway: The server, acting as a gateway or proxy, received an invalid response from an upstream server.
- 503 Service Unavailable: The server cannot handle the request (usually due to maintenance or overloading).
- 504 Gateway Timeout: The server, acting as a gateway, did not receive a timely response from an upstream server.

HTTP Request-Response Cycle Working Process

1. User enters URL
2. Browser sends HTTP request
3. Server processes request
4. Database interaction occurs if needed
5. Server sends HTTP response
6. Browser displays webpage

Client-side & Server-side Scripting

Client-side Scripting

- Executed in the user's browser.

Languages

- HTML
- CSS
- JavaScript

Advantages

- Fast response

- Better user experience
- Reduces server load

Example

- Form validation in:
- eSewa login page
- Online admission forms

Client-side & Server-side Scripting

Server-side Scripting

- Executed on the web server.

Languages

- PHP
- Python
- Java
- Node.js
- ASP.NET

Functions

- Database connectivity
- Authentication
- Business logic

Example

- Online banking systems
- University result systems

Client-side vs Server-side Scripting

Client-side	Server-side
Runs in browser	Runs on server
Faster interaction	More secure
Uses JavaScript	Uses PHP/Python/Java
Visible to user	Hidden from user

DNS (Domain Name System)

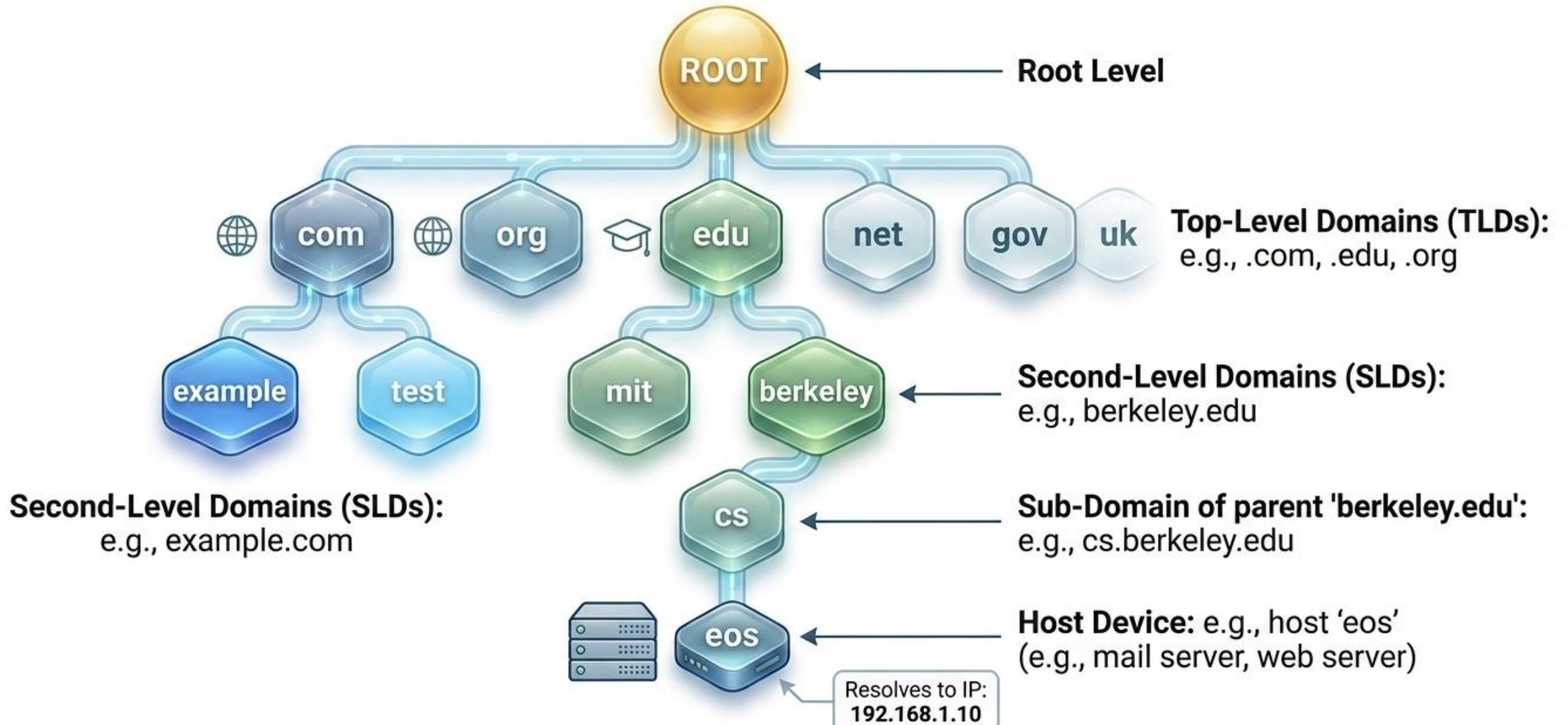
DNS

- Converts domain names into IP addresses.
- **Example: `www.esewa.com.np` → `103.x.x.x`**

Why DNS?

- Humans remember names easier than IP addresses.

DNS Hierarchy



Types of Domains in Nepal

Domain	Purpose
.com.np	Commercial
.edu.np	Educational
.gov.np	Government
.org.np	Organization

Web Hosting

Web Hosting Service that stores websites on servers connected to the Internet.

Types of Hosting

- Shared Hosting
- VPS Hosting
- Dedicated Hosting
- Cloud Hosting

Hosting Providers in Nepal

Nepal Hosting Providers

- [Mercantile Communications](#)
- [Himalayan Host](#)
- [Nest Nepal](#)

Static vs Dynamic Website Hosting

Static Website	Dynamic Website
HTML/CSS only	Uses database
Faster	Interactive
Easier hosting	Requires backend

Examples

Static

- College notice website

Dynamic

- eSewa, Daraz, Merojob

Security Considerations: Web Security Basics

Important Measures:

- Use HTTPS
- Strong passwords
- SSL certificates
- Firewall
- Regular updates

Protection against:

- Online fraud
- Data theft
- Banking attacks

Next Chapter....